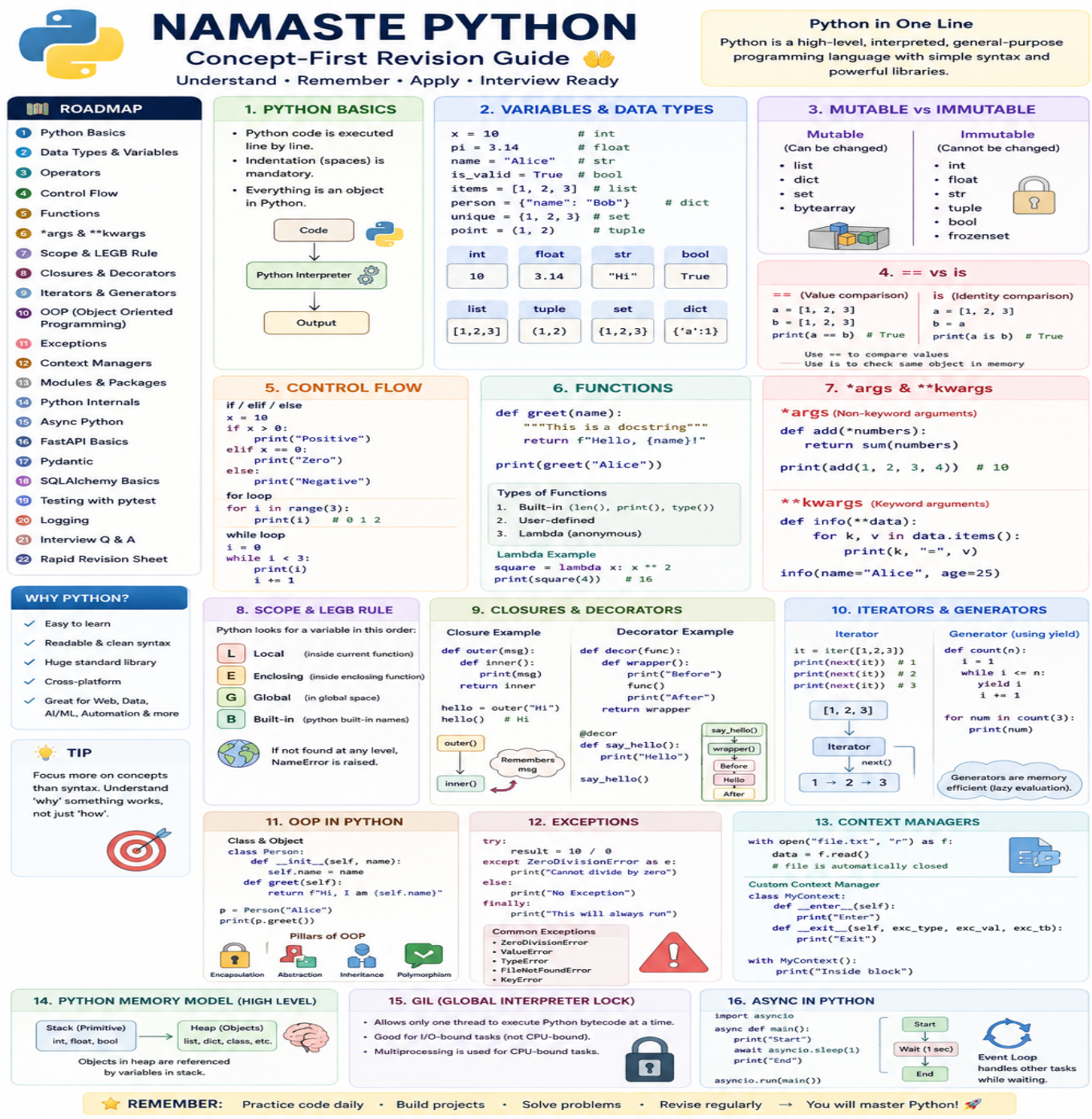
Namaste Python

Visual & Easy-to-Understand Revision Guide

The goal is not to memorize syntax. Understand the mental model first, then use the code as a reminder.



1. How Python Thinks

In simple words: Python variables are names pointing to objects. The object lives in memory; the variable gives you a way to reach it.

Mental model: name → object → value/type. Assignment usually changes what a name points to.

See it in code

```
a = [1, 2]
b = a
b.append(3)
print(a) # [1, 2, 3]
```

Remember: When two names point to the same mutable object, changing it through one name is visible through the other.

Interview question: What is the difference between a variable, an object, and a reference?

2. Mutable vs Immutable

In simple words: Mutable objects can change in place. Immutable objects cannot; a new object is created when you appear to change the value.

Mental model: list/dict/set → usually mutable. int/float/str/tuple/bool → immutable.

See it in code

```
items = [1, 2]
items.append(3) # mutation

name = 'Sam'
name = name + ' Hazra' # rebinding
```

Remember: Mutation changes an existing object. Rebinding makes a name point somewhere else.

Interview question: Why can a tuple contain a mutable list even though the tuple itself is immutable?

3. == vs is

In simple words: `==` asks: do these objects have equal values? `is` asks: are these the exact same object?

Mental model: Think: value comparison vs identity comparison.

See it in code

```
a = [1, 2]
b = [1, 2]
print(a == b) # True
print(a is b) # False
```

Remember: Use `is` for identity checks such as `value is None`.

Interview question: Why should you normally use `is None` instead of `== None`?

4. Functions → Scope → Closure → Decorator

In simple words: These concepts build on each other. A function can access names from enclosing scopes; a closure remembers them; a decorator uses this ability to wrap functions.

Mental model: Function → nested function → captured state → wrapper.

See it in code

```
def multiplier(n):
def multiply(x):
return x * n
return multiply

double = multiplier(2)
print(double(5)) # 10
```

Remember: A closure remembers state from the enclosing scope.

Interview question: What is a closure, and how is it related to a decorator?

5. Iterable → Iterator → Generator

In simple words: An iterable is something you can loop over. An iterator produces one item at a time. A generator is an easy way to create an iterator using `yield`.

Mental model: for loop → iter() → next() → StopIteration.

See it in code

```
def numbers():
yield 1
yield 2
yield 3

for n in numbers():
print(n)
```

Remember: Generators are lazy: values are produced only when needed, which is useful for large data.

Interview question: What is the difference between an iterable, an iterator, and a generator?

6. OOP: Four Core Ideas

In simple words: Encapsulation groups data and behavior. Inheritance reuses/extends behavior. Polymorphism lets different objects respond to the same interface.

Mental model: Class = blueprint. Object = instance. Method = behavior attached to an object/class.

See it in code

```
class User:
def __init__(self, name):
self.name = name

def greet(self):
return f'Hi {self.name}'
```

Remember: Know `self`, `__init__`, instance vs class attributes, inheritance, MRO, and `super()`.

Interview question: Explain instance method vs classmethod vs staticmethod.

7. Exceptions

In simple words: Exceptions are Python's way to signal that normal execution cannot continue as expected.

Mental model: try → exception? → except → optional else → finally cleanup.

See it in code

```
try:
value = int('abc')
except ValueError:
value = 0
finally:
print('cleanup')
```

Remember: Catch specific exceptions. Do not hide errors with a broad bare `except`.

Interview question: What is the difference between `except`, `else`, and `finally`?

8. Context Managers

In simple words: A context manager makes resource handling safe and automatic.

Mental model: enter resource → use resource → guaranteed cleanup.

See it in code

```
with open('data.txt') as file:
data = file.read()
# file is closed automatically
```

Remember: Think `with` whenever you acquire something that must be released.

Interview question: How do `__enter__` and `__exit__` work?

9. Memory + GIL

In simple words: Python objects live in memory and names reference them. CPython manages memory with reference counting plus cyclic garbage collection.

Mental model: name → object in heap. Standard CPython's GIL historically limits concurrent execution of Python bytecode within one interpreter.

See it in code

```
a = []
b = a
# two names reference one object

del a
```

```
# b still references the object
```

Remember: Threading is commonly useful for I/O-bound work; multiprocessing is useful for CPU-bound parallel work. Async is another model for I/O concurrency.

Interview question: What is the GIL, and why doesn't it mean Python cannot do concurrency?

10. Async Python: The Big Picture

In simple words: Async Python lets one thread make progress on other tasks while an I/O operation is waiting.

Mental model: coroutine → event loop → await I/O → task pauses → another task runs → resume.

See it in code

```
async def fetch():
data = await get_data()
return data

results = await asyncio.gather(fetch(), fetch())
```

Remember: Do not put long blocking operations inside an async path unless you deliberately move them off the event loop.

Interview question: What exactly happens when Python reaches `await`?

11. FastAPI Mental Model

In simple words: A request enters the route, dependencies are resolved, input is validated, business logic runs, and the response is serialized.

Mental model: HTTP request → route → Depends → Pydantic validation → service → DB → response.

See it in code

```
from fastapi import Depends

@app.get('/me')
def me(user=Depends(get_current_user)):
return user
```

Remember: Keep authentication, DB sessions, and reusable request logic in dependencies/services rather than duplicating them.

Interview question: Why is FastAPI dependency injection useful?

12. SQLAlchemy Mental Model

In simple words: The Session is the unit that tracks ORM objects and manages transactions. Querying, flushing, committing, and rolling back are different operations.

Mental model: Python object → Session → SQL → database. flush sends SQL; commit finalizes the transaction.

See it in code

```
user = User(name='Sam')
session.add(user)
session.flush() # SQL sent
session.commit() # transaction committed
```

Remember: Learn N+1, lazy loading, joinedload/selectinload, transactions, and async sessions.

Interview question: What is the difference between flush and commit?

13. Testing

In simple words: A good test proves behavior and catches regressions without depending too heavily on implementation details.

Mental model: Arrange → Act → Assert.

See it in code

```
def test_add():
result = 2 + 3
assert result == 5
```

Remember: For FastAPI, know fixtures, dependency overrides, test clients, mocking, and database isolation.

Interview question: How would you test a FastAPI endpoint that depends on the current user?

14. Your Python Interview Map

In simple words: For interviews, move from fundamentals to internals and then to backend-specific design.

Mental model: Python basics → functions/scope → iterators/generators → OOP → memory/GIL → async → FastAPI → SQLAlchemy → testing.

Remember: If you can explain the mental model without looking at code, you are ready to revise the syntax.

Interview question: Be prepared to explain not just what something does, but why it behaves that way.

15. One-Page Final Revision

Topic	Remember this
Variables	Names point to objects
Mutable	Can change in place
Immutable	Cannot change in place
==	Value equality
is	Object identity
LEGB	Local → Enclosing → Global → Built-in
Closure	Inner function remembers enclosing state
Decorator	Wrap/extend callable behavior
Iterable	Can produce an iterator
Iterator	Produces next item + maintains state
Generator	Lazy iterator using yield
OOP	Class, object, inheritance, polymorphism
MRO	Method lookup order
Context manager	Safe setup + cleanup
GIL	CPython bytecode execution constraint
async/await	Cooperative I/O concurrency
Event loop	Schedules/resumes async tasks
FastAPI Depends	Reusable dependency graph
Pydantic	Validation + serialization
SQLAlchemy Session	ORM identity + transaction management
flush	Send pending SQL within transaction
commit	Finalize transaction
pytest	Arrange → Act → Assert

Revision rule: Read the visual → explain it aloud → write one example from memory → solve one interview question.